Resource Utilization (USE Method)

a checklist that finds the bottleneck fast

Phase 3 videos on YouTube

In one line: the USE method (Brendan Gregg): for every resource, hardware and software, check Utilization (how busy), Saturation (work waiting in a queue) and Errors; saturation is the key signal, because queues cause latency and averages hide bursts and single hot cores.

1 Where do you even start?

Latency is up and the dashboards are full of graphs. CPU looks fine. Memory looks fine. Guessing wastes hours. The **USE method** gives you a checklist that finds the bottleneck fast.

2 Utilization, saturation, errors

Created by Brendan Gregg, it says: **for every resource, check all three**.

- **Utilization**: how busy the resource is, as a share of time or capacity.
- **Saturation**: extra work that can't be served yet, usually waiting in a queue.
- **Errors**: failures, like dropped packets or disk errors. Often the quickest to check, so start there.

It works for hardware and for software resources: thread pools, connection pools and file descriptors all have a utilization, a queue and errors.

resource	Utilization	Saturation	Errors
CPU	% busy per core	run queue (vmstat r)	machine-check errors
Memory	% used	swapping, OOM kills	allocation failures
Disk	% busy (iostat %util)	queue length, await ms	I/O errors (dmesg)
Network	bandwidth used	drops, retransmits	interface errors
Pools (software)	active / max	waiters for a slot	timeouts acquiring

Figure 1. The USE checklist: what to look at for each resource.

3 Why saturation matters most

A resource at 70% *average* utilization can still be saturated in bursts, with requests queueing. Averages also hide individual cores: one core at 100% can bottleneck a single-threaded process while the machine average looks low. A disk at 40% busy can still have requests waiting behind a burst of writes. So look at **queues**, not just percentages (lesson 45 explains why queues explode).

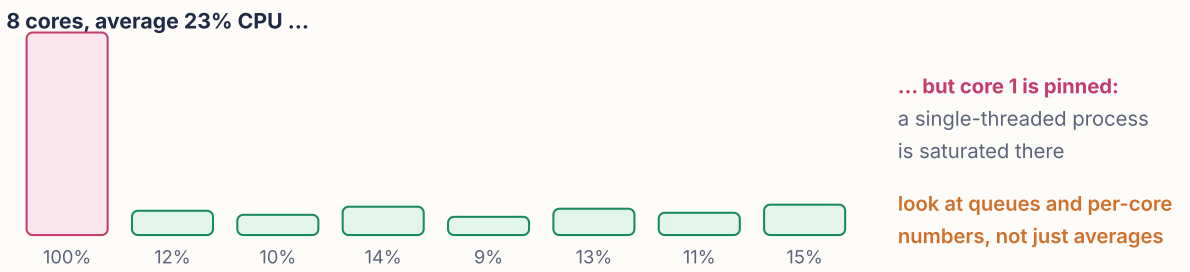


Figure 2. A healthy-looking average with one saturated core.

4 Tools

- `vmstat 1` and `top` (press 1 for per-core): CPU and the run queue (`r`).
- `free -m` and `vmstat` si/so: memory and swapping; `dmesg` for OOM kills.
- `iostat -x 1`: disk %util, queue size and await time.
- `sar -n DEV`, `ss -s`, `netstat -s`: network throughput, drops, retransmits.
- In the cloud, the same metrics come from CloudWatch or Azure Monitor (watch burst credits and IOPS limits on cloud disks).

WORKED EXAMPLE: LATENCY JUMPS, CPU IS FINE

API latency jumps. CPU is at 30% and memory is fine. Check the disk: `iostat` shows utilization near 100%, a long queue and wait times of 50 ms. Disk I/O is saturated. The cause: a log job writing to the same volume as the database. Move it to its own disk, and latency returns to normal.

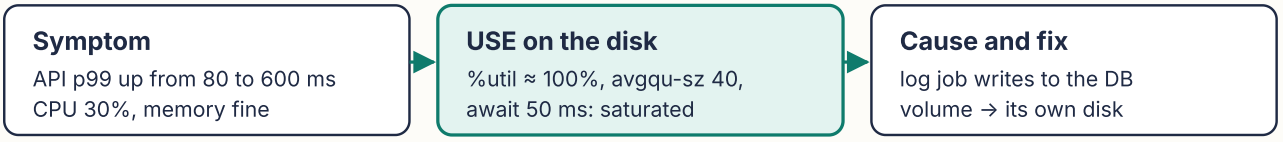


Figure 3. USE in action: walking the resources finds the saturated disk in minutes.

5 Use it systematically

The power of USE is that it's a checklist: you walk the list of resources and ask the three questions, so you won't miss the saturated one. Start with resources closest to the symptom, then work outward. It finds **resource** bottlenecks quickly; for problems inside the code itself you need profiling (next lesson).

COMMON MISTAKES

- Looking only at average CPU and memory.
- Forgetting software resources: a full connection pool looks like a slow database.
- Ignoring errors (retransmits, disk errors) that explain the latency directly.

6 Key terms

USE method: Utilization, Saturation, Errors for every resource

Utilization: how busy a resource is

Saturation: queued work waiting for the resource

Run queue: threads waiting for a CPU

await (iostat): average time an I/O request takes, including queueing

Software resource: pools, locks, file descriptors

7 Interview questions

Q1. What is the USE method? BASIC

For every resource, like CPU, memory, disk and network, check utilization, saturation and errors, to find the bottleneck systematically.

Q2. Why focus on saturation? BASIC

Because queueing causes latency, and averages can look healthy while a resource is saturated in bursts or on a single core.

Q3. Latency is high, CPU is 25%, memory fine. What next? SCENARIO

Walk USE: check per-core CPU, disk %util and await, network drops and retransmits, and software pools (DB connections, thread pools) for waiters and timeouts.

Q4. How would you apply USE to a database connection pool? SCENARIO

Utilization: connections in use / max. Saturation: requests waiting for a connection and wait time. Errors: timeouts acquiring a connection.

Q5. A disk shows 40% utilization but long waits. How? TRICKY

Utilization is averaged over the interval; bursts of writes can fill the queue briefly, and fast SSDs serve requests in parallel so %util understates load. Look at queue size and await.

Q6. When doesn't USE find the problem? TRICKY

When the bottleneck is in the code (an inefficient algorithm, lock contention inside the app) or in a downstream dependency; then use profiling and distributed tracing.

Next: SD-J-010 · Performance Profiling backendroadmap.com